

Operating Systems

Simple revision notes — concepts, definitions and key points

1. What is an Operating System?

An **Operating System (OS)** is system software that acts as an interface between the user and the computer hardware. It manages all hardware and software resources and provides an environment in which programs can run conveniently and efficiently. Examples: Windows, Linux, macOS, Android, iOS, Unix.

Main Functions

- **Process management** – creating, scheduling and terminating processes.
- **Memory management** – allocating and freeing main memory.
- **File management** – storing, naming, organising and protecting files.
- **Device / I/O management** – controlling devices through device drivers.
- **Security & protection** – user authentication and access control.
- **User interface** – CLI (command line) or GUI (graphical).

Types of Operating Systems

Type	Key idea
Batch OS	Similar jobs are grouped and executed in batches; no direct user interaction.
Multiprogramming	Several programs kept in memory; CPU switches when one waits for I/O.
Time-sharing (Multitasking)	CPU time divided into small slices so many users feel served at once.
Real-time OS (RTOS)	Guarantees response within a strict time limit (e.g. airbags, robotics).
Distributed OS	Many machines connected by a network appear as one system.
Mobile / Embedded OS	Lightweight OS for phones and dedicated devices (Android, iOS).

2. Process Management

A **program** is a passive set of instructions stored on disk. A **process** is a program in execution — it is active and has its own memory, registers and program counter.

Process States

- **New** – the process is being created.
- **Ready** – waiting to be assigned to the CPU.
- **Running** – instructions are being executed.
- **Waiting / Blocked** – waiting for an I/O or event to complete.
- **Terminated** – execution finished.

PCB (Process Control Block): a data structure holding all information about a process — process ID, state, program counter, CPU registers, memory limits, open files and priority. A **context switch** saves the PCB of the current process and loads the PCB of the next one.

Process vs Thread

Process	Thread
Independent program in execution.	Lightweight unit inside a process.
Has its own memory space.	Shares memory with other threads of the process.
Context switching is slow.	Context switching is fast.
Creation is costly.	Creation is cheap.

3. CPU Scheduling

Scheduling decides which ready process gets the CPU next. The goal is high CPU utilisation and throughput with low waiting time, turnaround time and response time.

- **Turnaround time** = Completion time – Arrival time
- **Waiting time** = Turnaround time – Burst time
- **Preemptive**: CPU can be taken away from a running process. **Non-preemptive**: it cannot.

Algorithm	Working	Note
FCFS	First come, first served (queue order).	Simple; convoy effect.
SJF	Shortest burst time first.	Optimal average waiting time.
SRTF	Preemptive version of SJF.	Burst time must be known.
Priority	Highest priority first.	Starvation; fix by ageing.
Round Robin	Each process gets a fixed time quantum.	Best for time-sharing.

4. Deadlock

A **deadlock** occurs when a set of processes are blocked forever because each holds a resource and waits for another resource held by some other process in the set.

Four Necessary Conditions (Coffman)

- **Mutual exclusion** – a resource can be used by only one process at a time.
- **Hold and wait** – a process holds one resource and waits for others.
- **No preemption** – a resource is released only voluntarily.
- **Circular wait** – a closed chain of processes each waiting for the next.

Handling methods: Prevention (break any one condition), Avoidance (*Banker's algorithm* – allocate only if the state stays safe), Detection and recovery, or simply Ignore it (the ostrich approach used by most general OSes).

5. Process Synchronisation

- **Race condition** – result depends on the order in which processes access shared data.
- **Critical section** – the part of code that accesses shared resources.
- A solution must satisfy **mutual exclusion**, **progress** and **bounded waiting**.
- **Semaphore** – an integer variable used with *wait(P)* and *signal(V)*; binary semaphore = mutex, counting semaphore = many instances.
- Classic problems: Producer–Consumer, Readers–Writers, Dining Philosophers.

6. Memory Management

- **Contiguous allocation** – each process gets one continuous block. Strategies: First Fit, Best Fit, Worst Fit.
- **Internal fragmentation** – unused space *inside* an allocated block.
- **External fragmentation** – free memory exists but is scattered in small pieces.
- **Paging** – memory is divided into fixed-size *frames* and the process into equal-size *pages*; removes external fragmentation.
- **Segmentation** – memory divided into logical variable-size segments (code, stack, data).
- **TLB** – a fast cache that stores recent page-table entries.

Virtual Memory

Virtual memory lets a process execute even if it is not fully in main memory. Pages are brought in on demand (**demand paging**). A **page fault** occurs when a required page is not in memory. Excessive page faults cause **thrashing**, where the CPU spends most time swapping instead of executing.

Page replacement algorithms: FIFO (oldest page out; suffers Belady's anomaly), Optimal (replace the page not needed for the longest time — a theoretical best), and LRU (replace the least recently used page).

7. File and Disk Management

- A **file** is a named collection of related information stored on secondary storage.
- **Access methods:** sequential, direct (random) and indexed.
- **Allocation methods:** contiguous, linked and indexed allocation.
- **Directory structures:** single-level, two-level, tree and acyclic-graph.
- **Disk scheduling:** FCFS, SSTF (shortest seek first), SCAN (elevator), C-SCAN, LOOK.

8. Quick Terms to Remember

Term	Meaning
Kernel	Core of the OS; always in memory and manages resources.
Shell	Interface that takes user commands and passes them to the kernel.
System call	Controlled way for a program to request a service from the kernel.
Bootstrapping	Loading the OS into memory when the computer starts.
Spooling	Buffering data (e.g. print jobs) so a slow device does not hold up the CPU.
Swapping	Moving a whole process between main memory and disk.
Multitasking	Running several tasks apparently at the same time.
Starvation	A process waits indefinitely because others are always preferred.

Tip: revise the tables and the four deadlock conditions first — they are the most frequently asked parts.